

2026

Institut National de l'audiovisuel

18 avenue des Frères

Lumière, 94360

Bry-sur-Marne



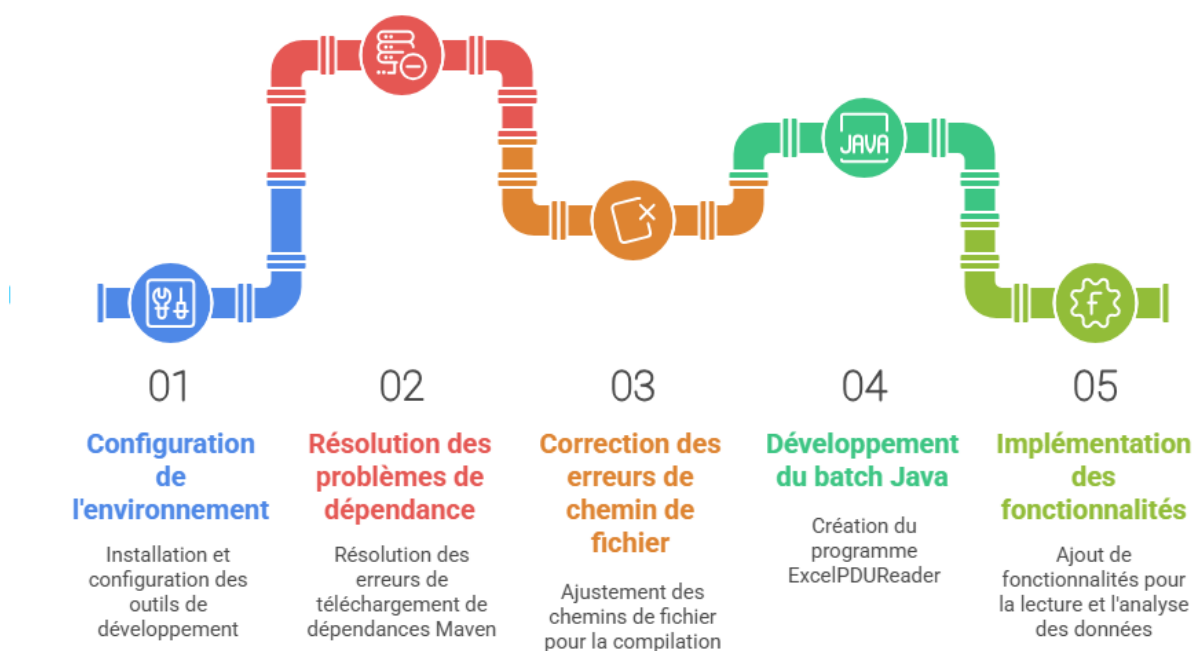
Développeur stagiaire

Stage effectué du 05/01/2026 au 13/02/2026

Tuteur de stage : Mounir Abid

Rapport de Stage - Semaine 1

Processus de développement de batch Java



Objectifs de la semaine

Le but principal de cette première semaine fut la prise en main d'un projet existant consistant en développement d'un batch Java capable de lire et analyser des fichiers Excel contenant les données de monitoring de PDU (Power Distribution Units) issues d'équipements de distribution électrique d'un datacenter, à l'origine, sans traitement automatisé pour faciliter leur exploitation.

Contexte du projet

Le projet consiste à analyser des fichiers Excel historiques dans lesquels des mesures électriques ont été relevées pour différents PDU répartis dans diverses baies du datacenter. Chaque fichier Excel référence plusieurs feuilles de calcul (A01, A02, etc.), et chacune de ces feuilles regroupe les mesures de 4 PDU identifiés par leur emplacement (1.1, 1.2, 2.1, 2.2).

Pour chaque PDU, les données suivantes sont enregistrées :

- Adresse IP
- Courant (A)
- Puissance apparente (VA)
- Facteur de puissance
- Tension (V)

Le fichier analysé durant cette semaine était : Histo_GT_DC_Bry1_19032023.xlsx

Travaux réalisés

1. Prise en main de l'environnement technique

Environnement : Ubuntu (WSL sur Windows)

Langage : Java

Gestionnaire de dépendances : Maven

Bibliothèque principale : Apache POI (version 5.2.3) pour la lecture de fichiers Excel

J'ai commencé par configurer l'environnement de développement :

- Installation et configuration de Maven
- Création de la structure de projet Maven standard
- Configuration du fichier pom.xml avec les dépendances nécessaires

2. Difficultés rencontrées et résolutions

Problème 1 : Téléchargement des dépendances Maven

Durant la première compilation avec mvn clean package, des erreurs de téléchargement sont apparues :

Failed to read artifact descriptor: Unexpected end of ZLIB input stream

Solution appliquée :

- Nettoyage du cache Maven corrompu : `rm -rf ~/.m2/repository/`
- Modification des versions des plugins Maven pour utiliser des versions plus stables (maven-compiler-plugin 3.8.1 au lieu de 3.11.0)
- Utilisation de l'option `-U` pour forcer la mise à jour des dépendances

Problème 2 : Chemin de fichier incorrect

Lors de la compilation avec javac, des erreurs "file not found" sont survenues car je me trouvais dans le mauvais répertoire.

Solution appliquée :

- Compréhension de la structure des chemins relatifs en environnement Linux/WSL
- Navigation vers la racine du projet (/mnt/c/Users/mgalli/excel-pdu-reader) avant compilation

- Création du répertoire target/classes pour stocker les fichiers compilés

3. Développement du batch Java

J'ai développé une première version du programme ExcelPDUReader.java avec les fonctionnalités suivantes :

Fonctionnalités implémentées :

- Ouverture et lecture d'un fichier Excel (.xlsx)
- Sélection d'un onglet spécifique (A01)
- Analyse automatique des en-têtes pour comprendre la structure des données
- Parcours des lignes de données
- Extraction et affichage des valeurs pour chaque PDU

Architecture du code :

ExcelPDUReader |—— main() : Point d'entrée, gestion du fichier Excel |—— analyzeHeaders()
: Analyse des en-têtes de colonnes |—— readPDUData() : Lecture des données de monitoring
|—— findDataStartRow() : Détection automatique de la ligne de début des données |——
readPDUValues() : Extraction des valeurs pour chaque PDU |—— getCellValueAsString() :
Conversion des cellules Excel en chaînes

4. Tests et résultats

Compilation réussie :

```
javac -cp "lib/*" src/main/java/ExcelPDUReader.java -d target/classes
```

Exécution réussie :

```
java -cp "target/classes:lib/*" ExcelPDUReader
...
```

****Résultat obtenu :****

Le programme a réussi à :

- Ouvrir le fichier Excel
- Lire l'onglet A01
- Détecter les en-têtes des colonnes
- Identifier les 4 PDU (1.1, 1.2, 2.1, 2.2)
- Extraire les adresses IP et les données de mesure

****Exemple de sortie :****

...

A01

=====

--- En-têtes détectés ---

Colonne C (index 2): PDU 1.1

Colonne D (index 3): 10.1.150.35

...

Heure : 1

PDU 1.1 : 1 - 1.26 - 300 - 0.85 - 230.4

PDU 1.2 : - - - -

PDU 2.1 : - - 1 - 0.24 - 55

PDU 2.2 : 0.94 - 230.7 - - -

Constats et observations

Points positifs

- Le programme fonctionne et arrive à lire le fichier Excel
- La bibliothèque Apache POI est bien adaptée pour cette tâche
- La structure du code est modulaire et facilement extensible

Travaux réalisés

1. Prise en main de l'environnement technique

Environnement : Ubuntu (WSL sur Windows)

Langage : Java

Gestionnaire de dépendances : Maven

Bibliothèque principale : Apache POI (version 5.2.3) pour la lecture de fichiers Excel

Configuration du projet Maven

J'ai d'abord créé le fichier pom.xml pour gérer les dépendances du projet :

```
<?xml version="1.0" encoding="UTF-8"?>
```

```

<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.pdu</groupId>
  <artifactId>excel-pdu-reader</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>11</maven.compiler.source>
    <maven.compiler.target>11</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <poi.version>5.2.3</poi.version>
  </properties>

  <dependencies>
    <!-- Apache POI pour lire les fichiers Excel -->
    <dependency>
      <groupId>org.apache.poi</groupId>
      <artifactId>poi</artifactId>
      <version>${poi.version}</version>
    </dependency>

    <dependency>
      <groupId>org.apache.poi</groupId>
      <artifactId>poi-ooxml</artifactId>
      <version>${poi.version}</version>
    </dependency>
  </dependencies>

```

</project>

2. Développement du batch Java

2.1 Structure principale et lecture du fichier Excel

J'ai développé la classe ExcelPDUReader avec une méthode main() qui gère l'ouverture et la fermeture sécurisée du fichier Excel :

```
import org.apache.poi.ss.usermodel.*;
import org.apache.poi.xssf.usermodel.XSSFWorkbook;

import java.io.File;
import java.io.FileInputStream;
import java.io.IOException;

public class ExcelPDUReader {

    public static void main(String[] args) {
        String filePath = "Histo_GT_DC_Bry1_19032023.xlsx";

        // Try-with-resources pour fermer automatiquement les ressources
        try (FileInputStream fis = new FileInputStream(new File(filePath));
            Workbook workbook = new XSSFWorkbook(fis)) {

            // Lecture de l'onglet A01
            Sheet sheet = workbook.getSheet("A01");

            if (sheet == null) {
                System.out.println("L'onglet A01 n'existe pas dans le fichier");
                return;
            }

            System.out.println("A01");
            System.out.println("=".repeat(80));
```

```

        // Lecture des en-têtes
        Row headerRow = sheet.getRow(0);
        if (headerRow != null) {
            analyzeHeaders(headerRow);
        }

        // Lecture des données
        readPDUData(sheet);

    } catch (IOException e) {
        System.err.println("Erreur lors de la lecture du fichier : " + e.getMessage());
        e.printStackTrace();
    }
}
}

```

Points techniques importants :

- Utilisation du try-with-resources pour éviter les fuites mémoires
- Vérification de l'existence de l'onglet avant traitement
- Gestion des exceptions IO pour les erreurs de fichier

2.2 Analyse des en-têtes

Cette méthode permet d'identifier automatiquement la structure du fichier Excel :

```

private static void analyzeHeaders(Row headerRow) {
    System.out.println("\n--- En-têtes détectés ---");

    for (int i = 0; i < headerRow.getLastCellNum(); i++) {
        Cell cell = headerRow.getCell(i);
        if (cell != null) {
            String value = getCellValueAsString(cell);
            if (!value.trim().isEmpty()) {

```



```

        // Affichage avec la lettre de colonne Excel et l'index numérique
        System.out.println("Colonne " + (char)('A' + i) +
            " (index " + i + "): " + value);
    }
}
}
System.out.println();
}
...

```

****Résultat obtenu :****

```

...
--- En-têtes détectés ---
Colonne C (index 2): PDU 1.1
Colonne D (index 3): 10.1.150.35
Colonne E (index 4): A01_Rouge
Colonne O (index 14): PDU 1.2
Colonne P (index 15): 10.1.150.35
...

```

2.3 Conversion des cellules Excel

Une des difficultés majeures était de gérer les différents types de données dans Excel (texte, nombre, date, formule). J'ai créé une méthode utilitaire pour uniformiser la conversion :

```

private static String getCellValueAsString(Cell cell) {
    if (cell == null) {
        return "";
    }

    switch (cell.getCellType()) {
        case STRING:
            return cell.getStringCellValue();

        case NUMERIC:

```

```

// Distinction entre date et nombre
if (DateUtil.isCellDateFormatted(cell)) {
    Date date = cell.getDateCellValue();
    SimpleDateFormat sdf = new SimpleDateFormat("HH:mm:ss");
    return sdf.format(date);
} else {
    double numValue = cell.getNumericCellValue();
    // Afficher les entiers sans décimales
    if (numValue == Math.floor(numValue)) {
        return String.valueOf((int) numValue);
    }
    return String.valueOf(numValue);
}

case BOOLEAN:
    return String.valueOf(cell.getBooleanCellValue());

case FORMULA:
    // Pour l'instant, retourne la formule brute
    return cell.getCellFormula();

case BLANK:
    return "";

default:
    return "";
}
}

```

Problème identifié : Les formules Excel sont retournées sous forme de texte (ex: IF(D3="KO"...)), pas les valeurs calculées. Cela nécessitera un correctif en Semaine 2.

2.4 Détection automatique du début des données

Le fichier Excel peut avoir des lignes vides ou des en-têtes multiples. J'ai implémenté une fonction de détection automatique :

```
private static int findDataStartRow(Sheet sheet) {  
    // Chercher la première ligne contenant des données numériques  
    for (int i = 0; i <= 5; i++) {  
        Row row = sheet.getRow(i);  
        if (row != null) {  
            Cell cell = row.getCell(1); // Colonne B (heure)  
            String value = getCellValueAsString(cell);  
  
            // Si on trouve une valeur qui contient un chiffre  
            if (value.matches(".*\\d+.*")) {  
                return i;  
            }  
        }  
    }  
    return 1; // Par défaut, ligne 1  
}
```

2.5 Lecture et extraction des données PDU

Cette méthode parcourt toutes les lignes de données et extrait les valeurs :

```
private static void readPDUData(Sheet sheet) {  
    int startRow = findDataStartRow(sheet);  
  
    for (int rowIndex = startRow; rowIndex <= sheet.getLastRowNum(); rowIndex++) {  
        Row row = sheet.getRow(rowIndex);  
        if (row == null) continue;  
  
        // Lecture de l'heure (colonne B)  
        Cell timeCell = row.getCell(1);  
        String heure = getCellValueAsString(timeCell);  
  
        if (heure.trim().isEmpty()) {
```

```

        continue;
    }

    System.out.println("Heure : " + heure);
    readPDUValues(row);
    System.out.println();
}
}

```

2.6 Extraction des valeurs pour chaque PDU

```

private static void readPDUValues(Row row) {
    String[] pduNames = {"PDU 1.1", "PDU 1.2", "PDU 2.1", "PDU 2.2"};

    int columnOffset = 2; // Commence à la colonne C
    int valuesPerPDU = 5; // IP + 4 valeurs de mesure

    for (int pduIndex = 0; pduIndex < pduNames.length; pduIndex++) {
        int baseColumn = columnOffset + (pduIndex * valuesPerPDU);

        String ip = getCellValueAsString(row.getCell(baseColumn));
        String courant = getCellValueAsString(row.getCell(baseColumn + 1));
        String puissance = getCellValueAsString(row.getCell(baseColumn + 2));
        String facteur = getCellValueAsString(row.getCell(baseColumn + 3));
        String tension = getCellValueAsString(row.getCell(baseColumn + 4));

        System.out.println("  " + pduNames[pduIndex] + " : " +
            ip + " - " +
            courant + " - " +
            puissance + " - " +
            facteur + " - " +
            tension);
    }
}

```

```
}  
...
```

3. Compilation et exécution

3.1 Structure du projet

```
...
```

```
excel-pdu-reader/
```

```
|— pom.xml  
|— src/  
|   |— main/  
|       |— java/  
|           |— ExcelPDUReader.java  
|— lib/                # JARs téléchargés manuellement  
|   |— poi-5.2.3.jar  
|   |— poi-ooxml-5.2.3.jar  
|   |— ...  
|— target/  
|   |— classes/  
|       |— ExcelPDUReader.class  
|       |— ExcelPDUReader$1.class  
|— Histo_GT_DC_Bry1_19032023.xlsx
```

3.2 Commandes de compilation

Avec Maven (tentative initiale) :

```
mvn clean package
```

```
# Créer le répertoire de destination
```

```
mkdir -p target/classes
```

```
# Compiler
```

```
javac -cp "lib/*" src/main/java/ExcelPDUReader.java -d target/classes
```

```
# Exécuter
```

```
java -cp "target/classes:lib/*" ExcelPDUReader
```

```
...
```

4. Tests et résultats

```
**Exemple de sortie obtenue :**
```

```
...
```

```
A01
```

```
=====
```

```
--- En-têtes détectés ---
```

```
Colonne C (index 2): PDU 1.1
```

```
Colonne D (index 3): 10.1.150.35
```

```
Colonne E (index 4): A01_Rouge
```

```
Colonne F (index 5): A01_Rouge_A
```

```
Colonne G (index 6): V20C2600242
```

```
Colonne O (index 14): PDU 1.2
```

```
Colonne P (index 15): 10.1.150.35
```

```
[...]
```

```
Heure : 1
```

```
PDU 1.1 : 1 - 1.26 - 300 - 0.85 - 230.4
```

```
PDU 1.2 : - - - -
```

```
PDU 2.1 : - - 1 - 0.24 - 55
```

```
PDU 2.2 : 0.94 - 230.7 - - -
```

```
Heure : 1
```

```
PDU 1.1 : 1 - 1.27 - 293 - 0.85 - 230.5
```

```
PDU 1.2 : - - - -
```

PDU 2.1 : - - 1 - 0.24 - 58

PDU 2.2 : 0.94 - 230.4 - - -

[...]

...

5. Difficultés rencontrées et résolutions

Problème 1 : Erreurs Maven

****Erreur rencontrée :****

...

Failed to read artifact descriptor for org.apache.maven.plugins:maven-compiler-plugin:jar:3.11.0

Unexpected end of ZLIB input stream

Diagnostic : Cache Maven corrompu, problème réseau lors du téléchargement des dépendances.

Solution appliquée :

Nettoyage du cache

rm -rf ~/.m2/repository/commons-cli

rm -rf ~/.m2/repository/org/apache/maven/plugins

Forcer la mise à jour

mvn clean package -U

Alternative finale : Téléchargement manuel des JARs et compilation avec javac.

Problème 2 : Chemins relatifs incorrects

Erreur :

\$ javac -cp "lib/*" src/main/java/ExcelPDUReader.java -d target/classes

error: file not found: src/main/java/ExcelPDUReader.java

Cause : J'étais positionné dans /mnt/c/Users/mgalli/excel-pdu-reader/src/main/java/ au lieu de la racine du projet.

Solution :

```
bash
```

```
# Revenir à la racine
```

```
cd /mnt/c/Users/mgalli/excel-pdu-reader
```

```
# Vérifier la position
```

```
pwd
```

```
# Compiler depuis la racine
```

```
javac -cp "lib/*" src/main/java/ExcelPDURReader.java -d target/classes
```

Leçon apprise : Toujours compiler depuis la racine du projet pour que les chemins relatifs fonctionnent correctement.

Constats et observations

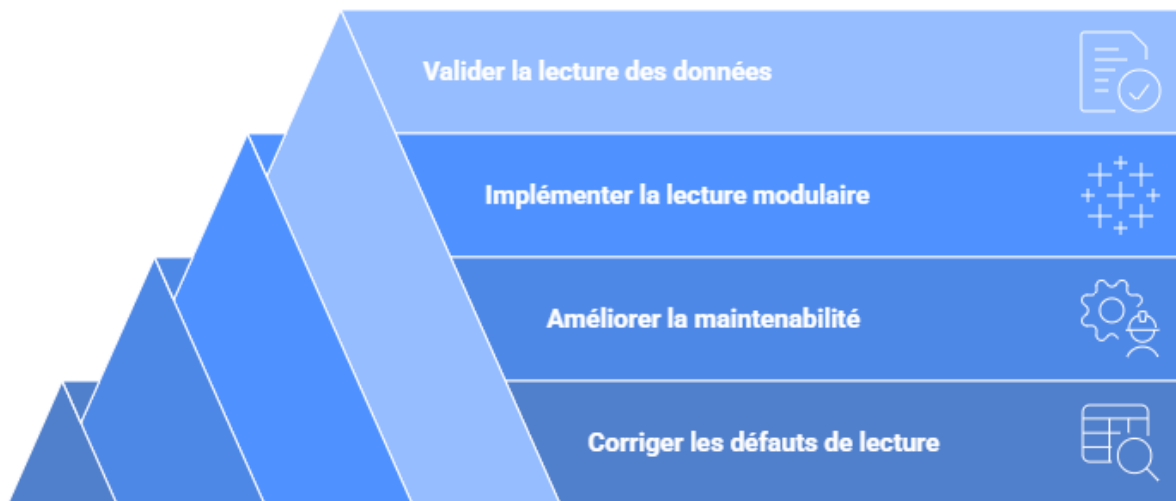
Points positifs

- Le programme fonctionne et arrive à lire le fichier Excel
- La structure du code est modulaire avec des méthodes bien séparées
- Bonne gestion des erreurs avec try-catch et try-with-resources
- Le code est documenté et compréhensible

Rapport de Stage - Semaine 2

Objectifs de la semaine

Atteindre des objectifs de refactoring de code



Comme la première semaine était consacrée à l'étude fonctionnelle puis ce deuxième cadrage a comme principal objectif le refactoring puis l'optimisation du code Java du batch pour

- Corriger les défauts de lecture des données identifiés la semaine 1
- Améliorer la maintenabilité du code par la suppression de la redondance
- Mettre en œuvre une lecture modulaire et configurable des PDU
- Valider la bonne lecture des données réelles du fichier Excel Rappel du contexte Ce projet est une étude de fichiers historiques Excel regroupant des mesures électriques pour des PDU (Power Distribution Units) d'un datacenter. Chaque feuille Excel (A01, A02, etc) présente les données de 4 PDU avec les mesures suivantes :
- Adresse IP
- Courant (A)
- Puissance apparente (VA)
- Facteur de puissance
- Tension (V)

Les fichiers analysés : Histo_GT_DC_Bry1_19032023.xlsx

Travaux réalisés 1. Analyse des problèmes de la semaine 1 Les défauts identifiés lors de la révision du code développé en semaine 1 sont les suivants : Défaut 1 : Mauvaise compréhension de la structure des données

- Constat : Les colonnes n'étaient pas espacées de 5 comme supposé initialement
- Conséquences : Les valeurs lues n'étaient pas dans les bonnes colonnes
- Exemple : Les IPs lues dans des cellules non conçues pour cela

Problème 2 : Code répétitif (anti-pattern)

```
System.out.println("PDU 1.1 : " + ip11 + " - " + courant11 + "...");  
System.out.println("PDU 1.2 : " + ip12 + " - " + courant12 + "...");  
System.out.println("PDU 2.1 : " + ip21 + " - " + courant21 + "...");  
System.out.println("PDU 2.2 : " + ip22 + " - " + courant22 + "...");
```

Problème : Violation du principe DRY (Don't Repeat Yourself)

Conséquence : Code difficile à maintenir et à étendre

Problème 3 : Positionnement des données

- Les adresses IP se trouvent à la **ligne 0** du fichier (ligne de métadonnées)
- Les en-têtes de colonnes se trouvent à la **ligne 1**
- Les données temporelles commencent à la **ligne 2**

2. Analyse approfondie de la structure du fichier

J'ai analysé en détail le fichier texte exporté (Histo_GT_DC_Bry1_19032023txt.txt) pour comprendre la vraie structure :

Structure identifiée :

Ligne	Contenu	Description
0	Métadonnées PDU	Noms, IPs et identifiants des PDU
1	En-tête colonne	heure", "Statut", "Courant (A)", etc.
2+	Données	Horodatages et mesures électriques

Observation importante : Les colonnes sont espacées de 12 positions (et non 5 comme supposé initialement).

Mapping des colonnes pour chaque PDU :

Caractéristique	PDU 1.1	PDU 1.2	PDU 2.1	PDU 2.2
Colonnes	02 rouge A	02 rouge b	02 bleu A	02 bleu B
Données	A	b	A	B
PDU	2	14	26	38
Plage	3-6	15-18	27-30	39-42

3. Refactorisation du code

3.1 Introduction d'une configuration des PDU

J'ai créé une constante de configuration pour centraliser les informations de chaque PDU :

```
// Configuration des PDUs : nom, colonne IP, colonne première valeur
private static final Object[][] CONFIG_PDU = {
    {"PDU 1.1", 2, 3},
    {"PDU 1.2", 14, 15},
    {"PDU 2.1", 26, 27},
    {"PDU 2.2", 38, 39}
};
```

Avantages de cette approche :

- Centralisation de la configuration
- Facilité d'ajout de nouveaux PDU
- Réduction des erreurs de typage
- Code plus lisible

3.2 Refactorisation de la lecture des IPs

Ancienne version (Semaine 1) :

```
Row pduInfoRow = sheet.getRow(0);
String ip11 = getCellValueAsString(pduInfoRow.getCell(2));
String ip12 = getCellValueAsString(pduInfoRow.getCell(14));
String ip21 = getCellValueAsString(pduInfoRow.getCell(26));
String ip22 = getCellValueAsString(pduInfoRow.getCell(38));
```

Nouvelle version (Semaine 2) :

```
// Lecture des IP des PDUs (ligne 0)
Row ligneInfosPDU = feuille.getRow(0);
String[] adressesIP = new String[CONFIG_PDU.length];

for (int i = 0; i < CONFIG_PDU.length; i++) {
    int colonneIP = (int) CONFIG_PDU[i][1];
    adressesIP[i] = convertirCelluleEnTexte(ligneInfosPDU.getCell(colonneIP));
}
```

Gain : Réduction de 4 lignes répétitives à une boucle générique

+ changement de noms de méthodes, variables en Français pour une meilleur compréhension

3.3 Refactorisation de la lecture des valeurs PDU

Ancienne version (Semaine 1) :

```
private static void readPDUValues(Row row, String ip11, String ip12,  
    String ip21, String ip22) {  
    // Bloc répétitif pour PDU 1.1  
    String courant11 = getCellValueAsString(row.getCell(3));  
    String puissance11 = getCellValueAsString(row.getCell(4));  
    // ... (répété 4 fois pour chaque PDU)  
    System.out.println("    PDU 1.1 : " + ip11 + " - " + ...);  
  
    // Bloc répétitif pour PDU 1.2  
    String courant12 = getCellValueAsString(row.getCell(15));  
    // ... (idem)  
  
    // Bloc répétitif pour PDU 2.1 et 2.2
```

Nouvelle version (Semaine 2) :

```
private static void lireValeursPDU(Row ligne, String[] adressesIP) {

    // Lecture des valeurs pour chaque PDU
    for (int i = 0; i < CONFIG_PDU.length; i++) {

        String nomPDU = (String) CONFIG_PDU[i][0];
        int colonneDebut = (int) CONFIG_PDU[i][2];
        String ip = adressesIP[i];

        String courant = convertirCelluleEnTexte(ligne.getCell(colonneDebut));
        String puissance = convertirCelluleEnTexte(ligne.getCell(colonneDebut + 1));
        String facteur = convertirCelluleEnTexte(ligne.getCell(colonneDebut + 2));
        String tension = convertirCelluleEnTexte(ligne.getCell(colonneDebut + 3));

        System.out.println("    " + nomPDU + " : " + ip + " - "
            + courant + " A - "
            + puissance + " VA - "
            + facteur + " - "
            + tension + " V");

    }

}
```

Gains :

- Réduction de ~40 lignes de code répétitif à une boucle de 15 lignes
- Extensibilité : ajout d'un 5ème PDU nécessite uniquement 1 ligne dans PDU_CONFIG
- Maintenabilité : modification du format d'affichage se fait en un seul endroit

3.4 Amélioration du formatage des nombres

J'ai modifié la méthode `getCellValueAsString()` pour améliorer l'affichage des nombres décimaux :

```
case NUMERIC:
    if (DateUtil.isCellDateFormatted(cellule)) {
        Date date = cellule.getDateCellValue();
        return new SimpleDateFormat("HH:mm:ss").format(date);
    } else {
        double valeur = cellule.getNumericCellValue();
        if (valeur == Math.floor(valeur)) {
            return String.valueOf((int) valeur);
        }
        // Nouveau : formatage à 2 décimales
        return String.format("%.2f", valeur);
    }
}
```

Avant : 0.25000001, 230.699999

Après : 0.25, 230.70

4. Tests et validation

4.1 Compilation et exécution

Compilation avec Maven

```
mvn clean package
```

Exécution

```
java -jar target/excel-pdu-reader-1.0-SNAPSHOT.jar Histo_GT_DC_Bry1_19032023.xlsx
```

4.2 Résultats obtenus

Sortie du programme (extrait) :

```
19/03/2023 23:58 :  
PDU 1.1 : 10.1.150.35 - 1.29 A - 295 VA - 0.85 - 230.2 V  
PDU 1.2 : 10.1.150.35 - 0.25 A - 58 VA - 0.93 - 230.4 V  
PDU 2.1 : 10.1.150.141 - 0.59 A - 136 VA - 0.68 - 231.5 V  
PDU 2.2 : 10.1.150.141 - 0.28 A - 65 VA - 0.9 - 231.5 V  
  
19/03/2023 23:43 :  
PDU 1.1 : 10.1.150.35 - 1.26 A - 300 VA - 0.85 - 230.4 V  
PDU 1.2 : 10.1.150.35 - 0.24 A - 55 VA - 0.94 - 230.7 V  
PDU 2.1 : 10.1.150.141 - 0.59 A - 137 VA - 0.68 - 231.5 V  
PDU 2.2 : 10.1.150.141 - 0.27 A - 63 VA - 0.9 - 231.5 V
```

Principes de développement employés

1. DRY (Don't Repeat Yourself)
Élimination de la répétition des 4 blocs PDU
Utilisation des boucles et de la configuration
2. Séparation des préoccupations
Configuration séparée de la logique métier
Méthodes dédiées à des tâches spécifiques
3. Maintenabilité
Code plus court et lisible
Ajouts de nouveaux PDU facilité
4. Robustesse
Gestion correcte des types des cellules Excel
Formatage cohérent des nombres

Difficultés rencontrées, résolutions cherchées

Difficulté 1 : Compréhension de la structure du fichier Problème :

La structure du fichier Excel était floue pour nous. Approche de résolution :

1. Exporter le fichier Excel en texte pour une étude
2. Identifier manuellement les colonnes relatives à chaque PDU
3. Contrôler grâce à plusieurs lignes de données
4. Documentation de la structure identifiée Leçon : Toujours analyser les données brutes avant de commencer à coder.

Compétences développées

Compétences techniques

1. Refactorisation de code
 - Identification des patterns répétitifs
 - Application du principe DRY
 - Amélioration de la lisibilité
2. Analyse de données
 - Compréhension de structures de fichiers complexes
 - Validation croisée des données
 - Documentation de formats
3. Java avancé
 - Manipulation de tableaux multidimensionnels
 - Utilisation de boucles génériques
 - Formatage de nombres avec `String.format()`
4. Apache POI
 - Lecture de métadonnées Excel
 - Navigation entre lignes et colonnes
 - Gestion des différents types de cellules

Compétences méthodologiques

1. Approche itérative
 - Amélioration progressive du code
 - Tests de validation après chaque modification
2. Documentation
 - Commentaires explicatifs dans le code
 - Documentation de la structure des données
3. Résolution de problèmes
 - Analyse méthodique des erreurs
 - Recherche de solutions élégantes

Rapport de Stage - Semaine 3

Les objectifs de la semaine

Après avoir élaboré la version alpha du batch lors de la Semaine 2, le travail de la Semaine 3 vise essentiellement à généraliser le batch pour :

Traiter tous les onglets du fichier Excel et pas seulement A01 ;

Mettre à disposition un système de filtrage avec un pattern de nommage ;

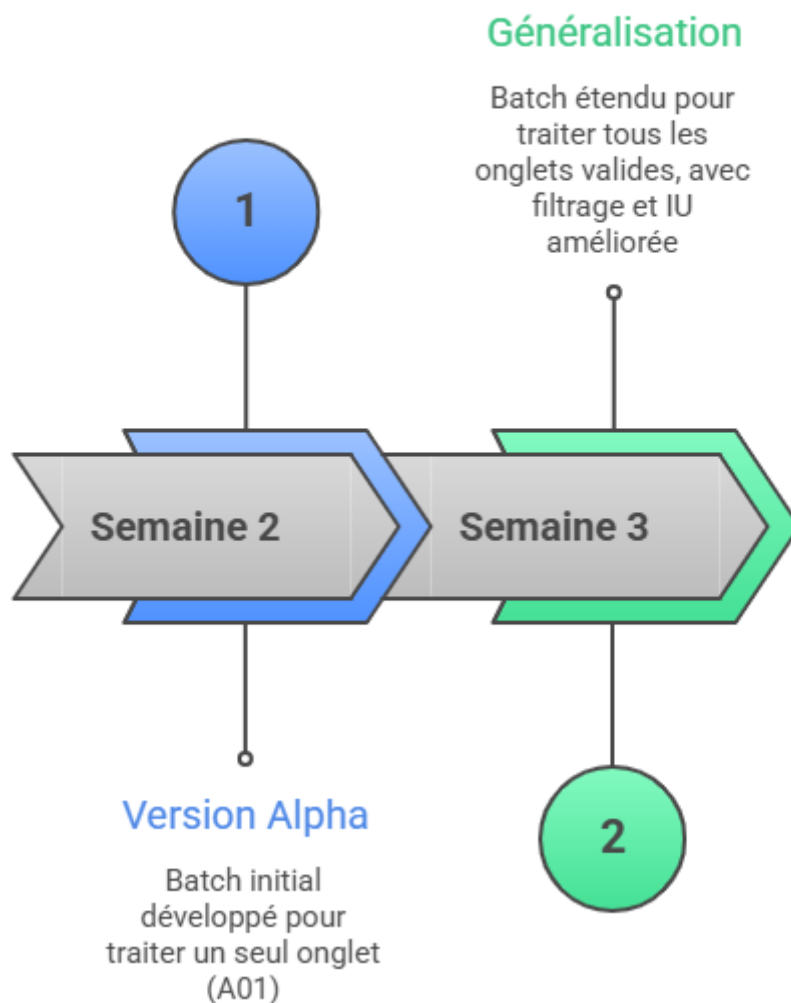
Augmenter les capacités d'interface utilisateur (IU) grâce à des séparateurs visuels et un résumé.

Fichier analysé : Histo_GT_DC_Bry1_19032023.xlsx

Rappel du contexte

Le batch n'était jusqu'alors capable de traiter qu'un seul onglet (A01) de manière fixe. Or, les fichiers Excel du datacenter contiennent plusieurs onglets relatifs à plusieurs baies (A01, A02, B01, C12, ...). Il est donc nécessaire de rendre le traitement automatique et générique.

Objectifs de la Semaine 3 pour le Batch Excel



Travaux réalisés

1. Analyse du besoin

Problème identifié : version (Semaine 2) :

```
// Récupération de l'onglet A01  
Sheet feuille = classeur.getSheet("A01");
```

Besoin exprimé :

- Traiter tous les onglets suivant le pattern : 1 lettre majuscule + 2 chiffres
- Exemples valides : A01, A02, B12, Z99
- Exemples à exclure : Sheet1, Données, ABC, A1, A123

2. Implémentation du système de filtrage par pattern

2.1 Import des expressions régulières

```
import java.util.regex.Pattern;
```

2.2 Définition du pattern de filtrage

```
// Pattern pour matcher une lettre + 2 chiffres (A01, B23, C99, etc.)  
private static final Pattern PATTERN_ONGLET = Pattern.compile("[A-Z]\\d{2}$");
```

Décryptage du pattern :

- `^` : début de la chaîne
- `[A-Z]` : une seule lettre majuscule (A à Z)
- `\\d{2}` : exactement 2 chiffres (00 à 99)
- `$` : fin de la chaîne

Tests de validation du pattern :

Comparaison des noms d'onglets

Nom d'onglet	Match ?	Raison
A01	✓	1 lettre + 2 chiffres
B23	✓	1 lettre + 2 chiffres
Z99	✓	1 lettre + 2 chiffres
A1	✗	1 seul chiffre
A123	✗	3 chiffres
ABC	✗	Pas de chiffres
Sheet1	✗	Plusieurs lettres

3. Refactorisation de la méthode main()

3.1 Parcours de tous les onglets

Ancienne version (Semaine 2) :

```
// Récupération de l'onglet A01
Sheet feuille = classeur.getSheet("A01");

if (feuille == null) {
    System.out.println("L'onglet A01 n'existe pas dans le fichier");
    return;
}

// Affichage du nom de l'onglet
System.out.println(feuille.getSheetName());
System.out.println("=".repeat(60));

// Lecture des données
lireDonneesPDU(feuille);
```

Nouvelle version (Semaine 3) :

```
// Compteur d'onglets traités
int nombreOngletsTraites = 0;

// Parcourir tous les onglets du classeur
for (int i = 0; i < classeur.getNumberOfSheets(); i++) {
    Sheet feuille = classeur.getSheetAt(i);
    String nomOnglet = feuille.getSheetName();

    // Vérifier si le nom correspond au pattern : 1 lettre + 2 chiffres
    if (PATTERN_ONGLET.matcher(nomOnglet).matches()) {

        // Séparateur visuel entre les onglets
        if (nombreOngletsTraites > 0) {
            System.out.println("\n\n" + "=".repeat(80));
            System.out.println("=".repeat(80) + "\n");
        }

        // Affichage du nom de l'onglet
        System.out.println(nomOnglet);
        System.out.println("=".repeat(60));

        // Lecture des données
        lireDonneesPDU(feuille);

        nombreOngletsTraites++;
    }
}
```

Gains :

- Automatisation complète du traitement
- Pas besoin de connaître les noms des onglets à l'avance
- Extensibilité : fonctionne avec n'importe quel fichier

3.2 Amélioration de l'affichage

Ajout de séparateurs visuels entre les onglets :

```
// Séparateur visuel entre les onglets
if (nombreOngletsTraites > 0) {
    System.out.println("\n\n" + "=".repeat(80));
    System.out.println("=".repeat(80) + "\n");
}
```

Ajout d'un résumé final :

```
// Affichage du résumé
System.out.println("\n\n" + "=".repeat(80));
System.out.println("Traitement terminé : " + nombreOngletsTraites + " onglet(s) traité(s)");
System.out.println("=".repeat(80));
```

4. Tests et validation

4.1 Exécution

mvn clean package

java -jar target/excel-pdu-reader-1.0-SNAPSHOT.jar Histo_GT_DC_Bry1_19032023.xlsx

A01

=====

19/03/2023 23:58 :

PDU 1.1 : 10.1.150.35 - 1.29 A - 295 VA - 0.85 - 230.20 V

PDU 1.2 : 10.1.150.35 - 0.25 A - 58 VA - 0.93 - 230.40 V

[...]

=====

=====

=====

=====

A02

=====

19/03/2023 23:58 :

PDU 1.1 : 10.1.150.36 - 1.15 A - 265 VA - 0.82 - 230.10 V

[...]

=====

=====

=====

=====

A03

=====

[...]

=====

=====

Traitement terminé : 12 onglet(s) traité(s)

Validation :

- Tous les onglets A01, A02, A03... traités automatiquement
- Séparation visuelle claire entre les onglets
- Résumé final informatif
- Les onglets "Sheet1" ou autres sont correctement ignorés

Comparaison des versions

Évolution du code

Comparaison des semaines

Critère	Semaine 2	Semaine 3	Amélioration
Onglets traités	1 seul (A01)	Tous les onglets valides	∞
Automatisation	Manuelle	Automatique	↑
Flexibilité	Aucune	Pattern configurable	↑
Expérience utilisateur	Basique	Améliorée (séparateurs + résumé)	↑

Compétences acquises

Compétences techniques :

Expressions régulières (Regex)

- Syntaxe et pattern(s)
- Validation de format(s)
- Méthode .matches()

Apache POI avancé

- Navigation multi-onglets
- getNumberOfSheets()
- getSheetAt(index)

Contrôle de flux

- Boucles for (comprenant des tests de conditions)

- Compteurs et états
- Test logique pour affichage

Compétences méthodologiques :

Généralisation du code

- Pivoter d'une solution spécifique vers une de type « générique »
- Anticiper les besoins futurs

Expérience utilisateur

Amélioration de la sortie (lisibilité)

- En contexte (informations / résumé)

Conclusion

Cette troisième semaine a permis de généraliser le batch pour traiter de façon automatique tous les onglets pertinents du fichier Excel. Le code est désormais :

Automatique : aucune intervention humaine pour rajouter les nouveaux onglets

Flexible : pattern de filtrage configurable

Robuste : ignore les onglets non pertinents

User-friendly : affichage structuré avec résumé

Le batch est à présent capable de traiter de façon entièrement automatique des fichiers Excel très complexes comportant une multitude d'onglets (plusieurs dizaines en tout).

Rapport de Stage - Semaine 4

Objectifs de la semaine

L'objectif principal de cette quatrième semaine consistait à incorporer la persistance des données dans une base SQLite, en suivant le modèle de la base actuelle Bright. L'objectif était de transférer les données historiques d'Excel vers une base de données structurée afin de regrouper toutes les informations de suivi.

Contexte : Le système Bright en cours d'utilisation se sert d'une base de données SQLite comportant trois tables (pdu, scan, oid_value). Le but est de transférer les données historiques conservées dans des fichiers Excel afin de regrouper l'intégralité de l'historique.

Comparaison des bases de données

Caractéristique	Base de données Bright	Base de données Excel
Type	SQLite	Fichiers Excel
Tables	pdu, scan, oid_value	Aucune
Données	Historique	Historique
Objectif	Regrouper toutes les informations de suivi	Regrouper toutes les informations de suivi

Travaux réalisés

1. Analyse du schéma de base Bright

Structure de la base SQLite cible :

-- Table des PDU (équipements)

```
CREATE TABLE pdu (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  ip TEXT,  
  location TEXT,  
  socket TEXT,  
  branch INTEGER,
```

```

    color TEXT,
    phase INTEGER,
    dc INTEGER
);

-- Table des scans (sessions de mesure)
CREATE TABLE scan (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    date TEXT,
    status TEXT
);

-- Table des valeurs OID (mesures SNMP)
CREATE TABLE oid_value (
    pdu_id INTEGER,
    scan_id INTEGER,
    oid TEXT,
    description TEXT,
    value REAL,
    FOREIGN KEY(pdu_id) REFERENCES pdu(id),
    FOREIGN KEY(scan_id) REFERENCES scan(id)
);

```

Un PDU = un dispositif matériel identifié grâce à son adresse IP et à son emplacement.

Un scan = une session de mesure horodatée.

Un OID = une mesure précise (par exemple, courant, tension) associée à un PDU et un scan.

2. Mapping des données Excel vers les OID SNMP

Les colonnes Excel ont été mappées vers des OID SNMP standard :

```
// Mapping OID
private static final Map<String, String> MAPPING_OID = new HashMap<>();
static {
    MAPPING_OID.put("courant", "1.3.6.1.4.1.534.6.6.7.6.1.1.3");
    MAPPING_OID.put("puissance", "1.3.6.1.4.1.534.6.6.7.6.1.1.4");
    MAPPING_OID.put("facteur", "1.3.6.1.4.1.534.6.6.7.6.1.1.5");
    MAPPING_OID.put("tension", "1.3.6.1.4.1.534.6.6.7.6.1.1.2");
}
```

3. Développement de l'intégration SQLite

3.1 Initialisation de la base de données

```
private static void initialiserBaseDeDonnees() throws SQLException {
    try {
        Class.forName("org.sqlite.JDBC");
        System.out.println("Driver SQLite chargé");
    } catch (ClassNotFoundException e) {
        System.out.println("Chargement auto du driver SQLite...");
    }

    connexionBD = DriverManager.getConnection("jdbc:sqlite:bright.db");
    System.out.println("Connexion BD établie");

    creerTables();
}
```

3.2 Création des enregistrements

Pour chaque ligne de données Excel :

1. Créer un scan avec l'horodatage

```
private static int creerScan(String dateHeure) throws SQLException {
    String sql = "INSERT INTO scan (date, status) VALUES (?, 'imported')";
```

2. Vérifier/créer le PDU dans la base

```
private static int obtenirOuCreerPDU(String ip, String location) throws SQLException {
    String sqlSelect = "SELECT id FROM pdu WHERE ip = ? AND location = ?";
    try (PreparedStatement pstmt = connexionBD.prepareStatement(sqlSelect)) {
        pstmt.setString(1, ip);
        pstmt.setString(2, location);
        ResultSet rs = pstmt.executeQuery();
        if (rs.next()) return rs.getInt("id");
    }

    String sqlInsert = "INSERT INTO pdu (ip, location) VALUES (?, ?)";
    try (PreparedStatement pstmt = connexionBD.prepareStatement(sqlInsert)) {
        pstmt.setString(1, ip);
        pstmt.setString(2, location);
        pstmt.executeUpdate();
    }
}
```

3. Insérer les valeurs OID pour chaque mesure

```
private static void insererOidValue(int pduId, int scanId, String oid, String description, String valeur) throws SQLException {
    if (valeur == null || valeur.trim().isEmpty()) return;

    String sql = "INSERT INTO oid_value (pdu_id, scan_id, oid, description, value) VALUES (?, ?, ?, ?, ?)";
    try (PreparedStatement pstmt = connexionBD.prepareStatement(sql)) {
        pstmt.setInt(1, pduId);
        pstmt.setInt(2, scanId);
        pstmt.setString(3, oid);
        pstmt.setString(4, description);
        pstmt.setDouble(5, Double.parseDouble(valeur));
        pstmt.executeUpdate();
    } catch (NumberFormatException e) {
        // Ignorer
    }
}
```

4. Gestion des dépendances Maven

Ajout de la dépendance SQLite dans pom.xml :

```
<!-- SQLite JDBC Driver -->
<dependency>
    <groupId>org.xerial</groupId>
    <artifactId>sqlite-jdbc</artifactId>
    <version>3.44.1.0</version>
</dependency>
```

Configuration du plugin Shade pour créer un JAR exécutable autonome :

```
<!-- Plugin pour créer un JAR avec toutes les dépendances -->
<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-shade-plugin</artifactId>
    <version>3.5.0</version>
    <executions>
        <execution>
            <phase>package</phase>
            <goals>
                <goal>shade</goal>
            </goals>
            <configuration>
                <transformers>
                    <transformer implementation="org.apache.maven.plugins.shade.resource.ManifestResourceTransformer">
                        <mainClass>com.pdu.ExcelPDURReader</mainClass>
                    </transformer>
                    <transformer implementation="org.apache.maven.plugins.shade.resource.ServicesResourceTransformer"/>
                </transformers>
                <filters>
                    <filter>
                        <artifact>*</artifact>
                        <excludes>
                            <exclude>META-INF/*.SF</exclude>
                            <exclude>META-INF/*.DSA</exclude>
                            <exclude>META-INF/*.RSA</exclude>
                        </excludes>
                    </filter>
                </filters>
            </configuration>
        </execution>
    </executions>
</plugin>
</plugins>
```

5. Résolution des problèmes techniques

Problème 1 : Driver SQLite non trouvé dans le JAR

Symptôme :

No suitable driver found for jdbc:sqlite:bright.db

Cause : Le plugin Maven Shade n'incluait pas correctement les services JDBC.
Solution : Ajout du `ServicesResourceTransformer` dans la configuration Shade pour inclure les métadonnées JDBC.

Problème 2 : `RETURN_GENERATED_KEYS` non supporté
Symptôme :

not implemented by SQLite JDBC driver

Avant :

```
PreparedStatement pstmt = connexionBD.prepareStatement(sql,  
Statement.RETURN_GENERATED_KEYS);  
pstmt.executeUpdate();  
ResultSet rs = pstmt.getGeneratedKeys();
```

Après :

```
try (PreparedStatement pstmt = connexionBD.prepareStatement(sql)) {
    pstmt.setString(1, dateHeure);
    pstmt.executeUpdate();

    // Récupérer l'ID avec last_insert_rowid()
    try (Statement stmt = connexionBD.createStatement();
         ResultSet rs = stmt.executeQuery("SELECT last_insert_rowid()")) {
        if (rs.next()) return rs.getInt(1);
    }
}
```

Problème 3 : Erreurs de syntaxe XML dans pom.xml

Symptôme :

Non-readable POM: expected end tags </plugins></build></project>

Cause : Balises XML mal fermées ou manquantes.

Solution : Vérification et correction complète du fichier pom.xml avec toutes les balises fermées correctement.

6. Tests et validation

6.1 Exécution

mvn clean package

java -jar target/excel-pdu-reader-1.0-SNAPSHOT.jar

Histo_GT_DC_Bry1_19032023.xlsx

6.2 Résultats

Sortie console (exemple)

```
Driver SQLite chargé
Connexion BD établie
Tables créées/vérifiées
```

```
Nombre total d'onglets : 104
```

```
=====
Onglet 1/104 : A01
=====
```

```
2023-03-19 23:28:02 :
```

```
PDU 1.1 : 10.1.150.35 - 0.85A - 230.50VA - - V
PDU 1.2 : 10.1.150.35 - 0.94A - 230.40VA - - V
PDU 2.1 : 10.1.150.141 - 0.69A - 231.40VA - - V
PDU 2.2 : 10.1.150.141 - 0.92A - 231.40VA - - V
```

Fichier Excel



ExcelPDUReader.java

- |— Lecture des onglets
- |— Extraction des données
- |— Mapping OID
- |— Insertion dans SQLite



bright.db

- |— Table pdu (équipements)
- |— Table scan (sessions)
- |— Table oid_value (mesures)

Compétences acquises :

Aspect technique

Bases de données relationnelles

- Conception de schémas avec relations entre tables
- Écriture de requêtes SQL sécurisées
- Gestion des transactions pour garantir la cohérence des données

Intégration avec SQLite via JDBC

- Configuration et connexion à la base
- Adaptation aux spécificités de SQLite
- Gestion des erreurs liées au driver

Gestion de projet avec Maven

- Création d'un exécutable unique (uber-JAR)
- Maîtrise des dépendances
- Personnalisation du packaging

Débogage

- Analyse des erreurs pour en comprendre l'origine
- Tests progressifs et validation des étapes
- Vérification systématique des données

Méthodologie :

Résolution de problèmes

- Recherche de la cause première des dysfonctionnements
- Exploration de différentes pistes de solution
- Validation des correctifs par des tests ciblés

Migration de données

- Conversion d'un format Excel vers une base SQL
- Assurance de l'intégrité et de la complétude des informations
- Vérification du résultat après l'import

Cette quatrième semaine a marqué une étape clé : l'intégration de la persistance des données. L'application est désormais capable de :

- Lire les archives Excel historiques
- Convertir les données au format Bright
- Les charger dans une base SQLite centralisée
- Traiter automatiquement l'ensemble des feuilles de calcul
- Gérer les erreurs et les cas particuliers

Les données historiques sont maintenant consolidées dans la base Bright, permettant une analyse unifiée avec les informations en temps réel du système de monitoring.

RAPPORT DE STAGE

Semaine 5

Migration et optimisation d'un batch Spring

Traitement de fichiers Excel pour données PDU

1.1 Cette cinquième semaine de stage, j'ai travaillé sur la migration et l'optimisation d'un batch de traitement de données PDU (Power Distribution Unit) vers Spring Batch. L'objectif du projet est assez clair : automatiser le traitement de multiples fichiers Excel qui contiennent des mesures électriques, pour ensuite les stocker dans une base de données.

1.2 Pour te donner un peu de contexte, l'entreprise collecte des données de mesures électriques – comme le courant, la puissance ou la tension – à partir de plusieurs PDU répartis dans les datacenters. Ces données sont historiquement stockées dans des fichiers Excel, accessibles via un partage réseau. Le défi était donc de mettre en place un système capable de traiter automatiquement tous ces fichiers pour alimenter une base de données centralisée.

Cette semaine, mes objectifs étaient les suivants :

- Migrer le batch existant vers Spring Batch, pour profiter de ses fonctionnalités avancées.
- Mettre en place un système de traitement par répertoire, avec un argument en ligne de commande pour plus de flexibilité.
- Optimiser la lecture des fichiers Excel, afin d'éviter les problèmes de mémoire.
- Ajouter la possibilité de redémarrer le traitement en cas d'interruption.
- Enfin, créer des scripts et une documentation pour faciliter l'utilisation du batch.

Travaux réalisés

2.1 Architecture Spring Batch

J'ai structuré le projet selon le pattern Spring Batch classique avec une séparation claire des responsabilités :

Composant	Rôle
ExcelPDUReader	Lecture lazy des fichiers Excel, traitement fichier par fichier pour optimiser la mémoire
MesurePDUProcessor	Transformation et validation des données avant insertion
FlatteningItemWriter	Écriture des données dans la base de données en mode batch

2.2 Gestion des arguments en ligne de commande

J'ai développé la classe BatchArgumentsConfig qui permet de récupérer le répertoire source passé en argument de commande. Cela offre une grande flexibilité pour pointer vers différents répertoires selon les besoins :

- Répertoire local pour les tests
- Partage réseau Windows (\\srv-europe\DSI\...) pour la production
- Montage WSL pour l'environnement de développement Ubuntu

2.3 Optimisation de la lecture Excel

Le ExcelPDUReader implémente l'interface ItemStreamReader pour permettre :

- Lecture lazy : Un seul fichier chargé en mémoire à la fois
- Traitement séquentiel : Parcours fichier par fichier, feuille par feuille, ligne par ligne

- Support du restart : Sauvegarde de la position (fichier, feuille, ligne, PDU) dans l'ExecutionContext
 - Logs détaillés : Affichage du progrès et de la liste complète des fichiers traités
- ### 2.4 Scripts et automatisation

Pour faciliter l'utilisation du batch, j'ai créé plusieurs scripts shell :

Script	Fonction
run-batch.sh	Menu interactif pour sélectionner le répertoire source (local, réseau, personnalisé)
diagnose.sh	Diagnostic complet du projet : structure, dépendances, compilation
test-config.sh	Vérification rapide de la configuration (Java, Maven, fichiers)

3. Technologies et outils utilisés

3.1 Framework et librairies

Technologie	Version	Usage
Spring Boot	2.x / 3.x	Framework principal
Spring Batch	5.x	Orchestration du batch
Apache POI	5.2.3	Lecture fichiers Excel
H2 Database	2.x	Base de données (dev)
SLF4J/Logback	2.x	Logging

3.2 Environnement de développement

- IDE : IntelliJ IDEA / VS Code
- Système : WSL Ubuntu 24 sous Windows
- Build : Maven 3.x
- Version Control : Git

4. Les défis rencontrés et comment nous les avons surmontés

4.1 Gestion de la mémoire

****Le problème :**** Au départ, charger tous les fichiers Excel en mémoire d'un coup provoquait des erreurs `OutOfMemoryError`, surtout avec les fichiers historiques qui pouvaient contenir plusieurs milliers de lignes. C'était un vrai casse-tête.

****La solution :**** Nous avons opté pour une lecture « paresseuse » (lazy) grâce à un `ItemStreamReader`. Concrètement, on ouvre un seul fichier à la fois, on lit les données ligne par ligne, et on ferme le workbook une fois le fichier traité avant de passer au suivant. Cela a permis de soulager considérablement la mémoire.

4.2 Accès au partage réseau depuis WSL

****Le problème :**** Depuis WSL (Ubuntu), les chemins réseau Windows (du style `\\srv-europe\...\`)` n'étaient tout simplement pas accessibles. Cela compliquait l'accès aux fichiers sources.

****La solution :**** On a utilisé la commande `mount` avec le driver `drvfs`, spécifique à WSL, pour monter le partage réseau. Pour simplifier, on a automatisé cette étape dans un script (`run-batch.sh`). En parallèle, pour les tests, une alternative simple a été de copier les fichiers en local.

4.3 Erreurs de compilation avec Maven

****Le problème :**** Lors de l'ajout de `BatchArgumentsConfig`, on s'est heurté à des erreurs de compilation liées à des packages manquants ou incompatibles. Ça a ralenti le développement au début.

****La solution :**** Nous avons créé une version simplifiée de `BatchArgumentsConfig`, compatible avec Java 8 et supérieur. Pour dépanner plus vite, un petit script de diagnostic (`diagnose.sh`) a aussi été mis en place pour identifier rapidement les problèmes de configuration.

4.4 Gestion des redémarrages

****Le problème :**** Si le traitement était interrompu, il fallait pouvoir reprendre exactement là où on s'était arrêté, sans réinsérer les données déjà traitées. Un vrai défi pour garantir la fiabilité.

****La solution :**** On a implémenté les méthodes `open()`, `update()` et `close()` de l'`ItemStreamReader` pour sauvegarder la position exacte (fichier, feuille, ligne, PDU) dans l'`ExecutionContext` de Spring Batch. Comme ça, en cas de redémarrage, le batch reprend au bon endroit, tout simplement.

5. Résultats et livrables

5.1 Fichiers Java

- `ExcelPDUReader.java` - Reader optimisé avec lecture lazy et support restart
- `BatchArgumentsConfig.java` - Gestion des arguments CLI (+ version simplifiée)
- `MesurePDU.java` - Modèle de données
- `MesurePDUProcessor.java` - Transformation des données
- `FlatteningItemWriter.java` - Writer vers base de données

5.2 Scripts shell

- `run-batch.sh` - Script interactif de lancement avec menu

- `diagnose.sh` - Diagnostic complet du projet
 - `test-config.sh` - Vérification rapide de la configuration
- ### 5.3 Documentation
- `README.md` - Documentation complète du projet
 - `GUIDE_UTILISATION.md` - Guide d'utilisation détaillé avec exemples
 - `TROUBLESHOOTING_MAVEN.md` - Guide de résolution des problèmes de compilation
 - `application.properties.example` - Exemple de configuration complète
- ### 5.4 Fonctionnalités implémentées

Fonctionnalité	Statut
Traitement multi-fichiers	✓ Implémenté
Argument ligne de commande	✓ Implémenté
Lecture lazy (optimisation mémoire)	✓ Implémenté
Support restart	✓ Implémenté
Logs détaillés	✓ Implémenté
Scripts d'automatisation	✓ Implémenté
Documentation complète	✓ Implémenté

6. Ce que j'ai appris et développé

6.1 Compétences techniques

- **Spring Batch** : J'ai pris en main les concepts clés comme Reader/Processor/Writer, l'ItemStreamReader, l'ExecutionContext, et j'ai appris à gérer le redémarrage d'un job.
- **Apache POI** : J'ai manipulé des fichiers Excel en Java, en utilisant les classes XSSFWorkbook, Sheet, Row et Cell pour lire et traiter les données.
- **Optimisation mémoire** : J'ai mis en place des techniques pour éviter les saturations, comme la lecture paresseuse (lazy reading), une gestion rigoureuse des ressources et la fermeture systématique des flux.
- **Scripting shell** : J'ai créé des scripts bash interactifs, avec des menus, un peu de couleur pour la clarté, et des diagnostics automatisés pour faciliter le travail au quotidien.
- **WSL/Linux** : Je me suis familiarisé avec le montage de partages réseau Windows sous Linux et la gestion des chemins d'accès qui fonctionnent sur plusieurs environnements.

6.2 Compétences transversales

- **Documentation** : J'ai appris à rédiger une documentation technique claire et complète, que ce soit des README ou des guides pas à pas pour les utilisateurs.
- **Résolution de problèmes** : J'ai développé une méthode pour diagnostiquer et corriger des erreurs, qu'il s'agisse de bugs de compilation ou de problèmes de configuration plus subtils.
- **Conception logicielle** : J'ai travaillé sur une architecture modulaire, en veillant à bien séparer les responsabilités et à appliquer des patterns de conception adaptés.
- **Autonomie** : J'ai pris l'initiative de chercher mes propres solutions, de créer des scripts d'aide et de documenter mes travaux pour gagner en efficacité et en indépendance.

7. Conclusion et perspectives

7.1 Bilan de la semaine

Cette cinquième semaine a été particulièrement enrichissante et productive. J'ai réussi à migrer avec succès le batch de traitement PDU vers Spring Batch, en implémentant toutes les fonctionnalités demandées et en allant au-delà avec la création de scripts d'automatisation et d'une documentation complète.

Le projet est désormais prêt à être testé sur l'ensemble des fichiers historiques du partage réseau Bright. L'architecture mise en place garantit une bonne scalabilité et une maintenance facilitée grâce à la documentation fournie.

7.2 Points forts

- Architecture robuste avec gestion du restart et optimisation mémoire
- Solution flexible permettant de traiter n'importe quel répertoire via argument CLI
- Scripts d'automatisation facilitant grandement l'utilisation du batch
- Documentation complète et professionnelle (README, guides, troubleshooting)
- Gestion proactive des problèmes potentiels (diagnostic, version simplifiée)

7.3 Perspectives

Court terme :

- Tester le batch sur un échantillon de fichiers du partage réseau Bright
- Valider les performances et le temps de traitement sur de gros volumes
- Corriger les éventuels bugs identifiés lors des tests

Moyen terme :

- Implémenter un système de notification (email) en fin de traitement
- Ajouter des métriques de performance (temps par fichier, nombre de lignes/seconde)
- Créer un dashboard de monitoring du batch

Long terme :

- Migration vers une base de données de production (PostgreSQL)
- Planification automatique du batch (cron, scheduler Spring)
- Traitement parallèle multi-threads pour améliorer les performances

Cette semaine m'a permis de consolider mes connaissances en Spring Batch et de développer ma capacité à créer des solutions complètes, documentées et prêtes à l'emploi. Je suis satisfait du résultat obtenu et confiant pour la suite du stage.

Rapport de Stage – Semaine 6

Nom, Prénom : Galli Maxime

Objectifs de la semaine

Cette sixième semaine a été dédiée à la refactorisation et à l'amélioration du code du batch Java développé depuis le début du stage. L'objectif principal était de :

Rendre le code plus lisible et maintenable en appliquant les bonnes pratiques de développement (modularité, gestion des logs, séparation des responsabilités).

Il est préférable de versionner le projet sur GitLab pour avoir une sauvegarde centralisée et faciliter la collaboration entre les membres de l'équipe. GitLab offre ainsi un espace de stockage centralisé pour le projet, ce qui permet à chaque membre de l'équipe d'accéder aux dernières mises à jour et de travailler ensemble de manière efficace. En versionnant le projet sur GitLab, nous pouvons nous assurer que toutes les modifications sont sauvegardées et que nous pouvons facilement revenir en arrière si nécessaire. Cette approche favorise également la transparence et la responsabilité, car chaque contribution est enregistrée et peut être revue par les autres membres de l'équipe.

Améliorer la robustesse du traitement des fichiers Excel et des erreurs potentielles.

Travail réalisé

1. Refactorisation du code

Ajout d'un système de logs (SLF4J) pour remplacer les `System.out.println`, permettant un suivi plus précis de l'exécution et des erreurs.

Modularisation : Découpage des méthodes longues en sous-fonctions spécialisées (ex : `lireIPs`, `lireHeure`, `afficherEntetesDebug`).

Gestion des ressources : Utilisation systématique de `try-with-resources` pour la gestion des flux et connexions base de données.

Amélioration des messages d'erreur : Logs plus explicites et gestion des exceptions centralisée.

2. Exemple de code avant/après

Avant :

```
System.out.println("IP " + CONFIG_PDU[i][0] + " (col " + colonneIP + ") = " + adressesIP[i]);
```

Après :

```
LOGGER.info("IP {} (col {}) = {}", CONFIG_PDU[i][0], colIP, ips[i]);
```

3. Versionnage sur GitLab

- Création d'un dépôt GitLab pour le projet.
- Structuration du dépôt avec un .gitignore adapté (exclusion des fichiers de base de données, des logs, etc.).
- Effectuez des validations régulières avec des messages clairs qui reflètent les étapes de refactorisation.

Bilan et perspectives

Cette semaine a permis de stabiliser la base de code et de la rendre plus professionnelle, tout en préparant le terrain pour les prochaines évolutions. Le projet est désormais versionné et partagé, ce qui facilitera les futures mises à jour et corrections.